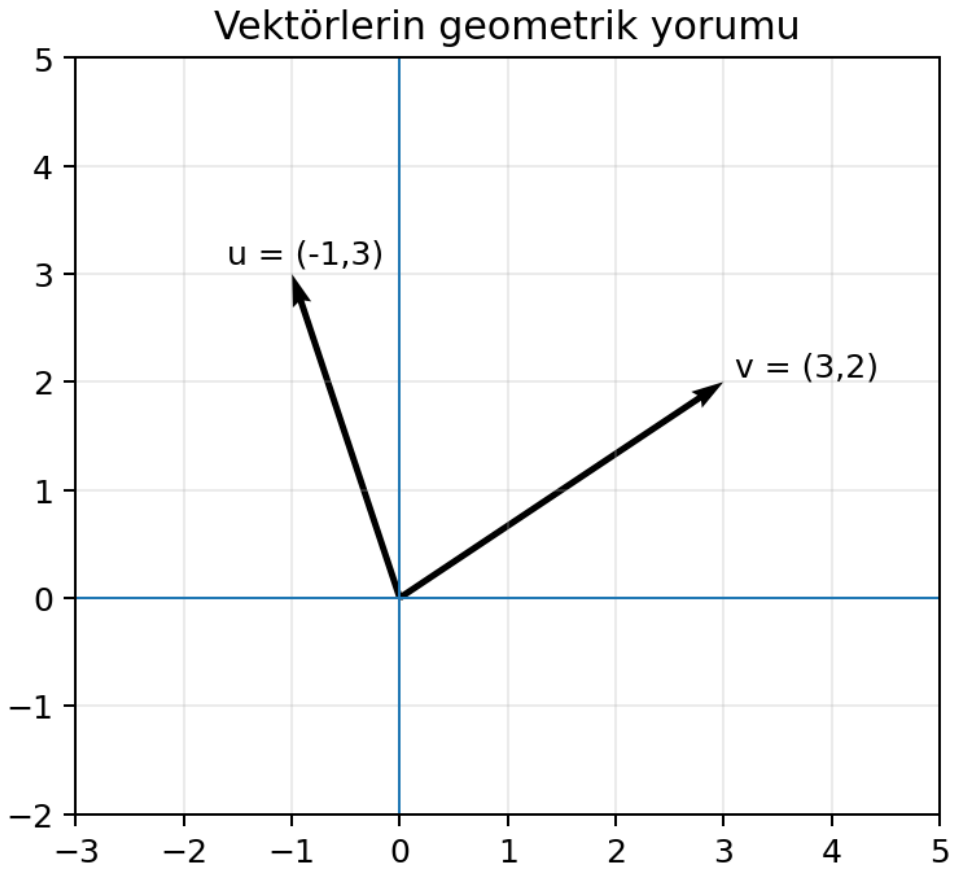
12. LINEER CEBİR

Veri Bilimci için Ayrıntılı Konu Anlatımı



Bu modülün amacı
Lineer cebiri soyut semboller toplamı olarak değil; veri setlerini, model katsayılarını, benzerliği, regresyonu, PCA'yı ve neural network hesaplarını anlamanın ortak dili olarak öğrenmek.

Kazanım	Bu modül sonunda
Vektör düşüncesi	Bir gözlemi veya feature setini uzayda nokta/yön olarak yorumlayabilirsin.
Matris düşüncesi	Bir veri tablosunu ve lineer dönüşümü matris olarak okuyabilirsin.
Geometri	Dot product, norm, açı, projection ve distance kavramlarını ML ile bağlayabilirsin.
Lineer cebir çekirdeği	Rank, inverse, determinant, eigenvalue/eigenvector ve SVD'nin anlamını kavrarsın.
ML bağlantısı	Linear regression, PCA, embeddings ve neural network katmanlarının matematiksel temelini görürsün.

1. Lineer cebir neden veri biliminde bu kadar önemli?

Modern veri biliminin büyük kısmı veriyi vektörler ve matrisler halinde temsil eder. Bir müşterinin yaşı, geliri ve harcaması; bir metnin embedding'i; bir görüntünün piksel değerleri; model katsayıları ve neural network ağırlıkları bu dilin farklı örnekleridir.

Veri bilimi nesnesi	Lineer cebir karşılığı
Tek müşteri kaydı	Feature vektörü x
Tüm veri seti	Design matrix X
Model katsayıları	Weight vector w
Tahmin	Xw gibi matris-vektör işlemi
Benzerlik	Dot product / cosine similarity
Boyut indirgeme	Eigenvectors / SVD / PCA
Neural network katmanı	Wx + b

Ana fikir
Lineer cebiri ezberlemek yerine her kavram için şu soruyu sor: 'Bu, veri üzerinde hangi geometrik veya hesaplama problemini çözüyor?'

2. Scalar, vector, matrix, tensor

Nesne	Örnek	Shape fikri
Scalar	42 veya 0.73	Tek değer
Vector	[age, income, spend]	(n,) veya (n,1)
Matrix	1000 müşteri x 20 feature	(1000,20)
Tensor	32 görüntü x 224 x 224 x 3	Çok boyutlu dizi

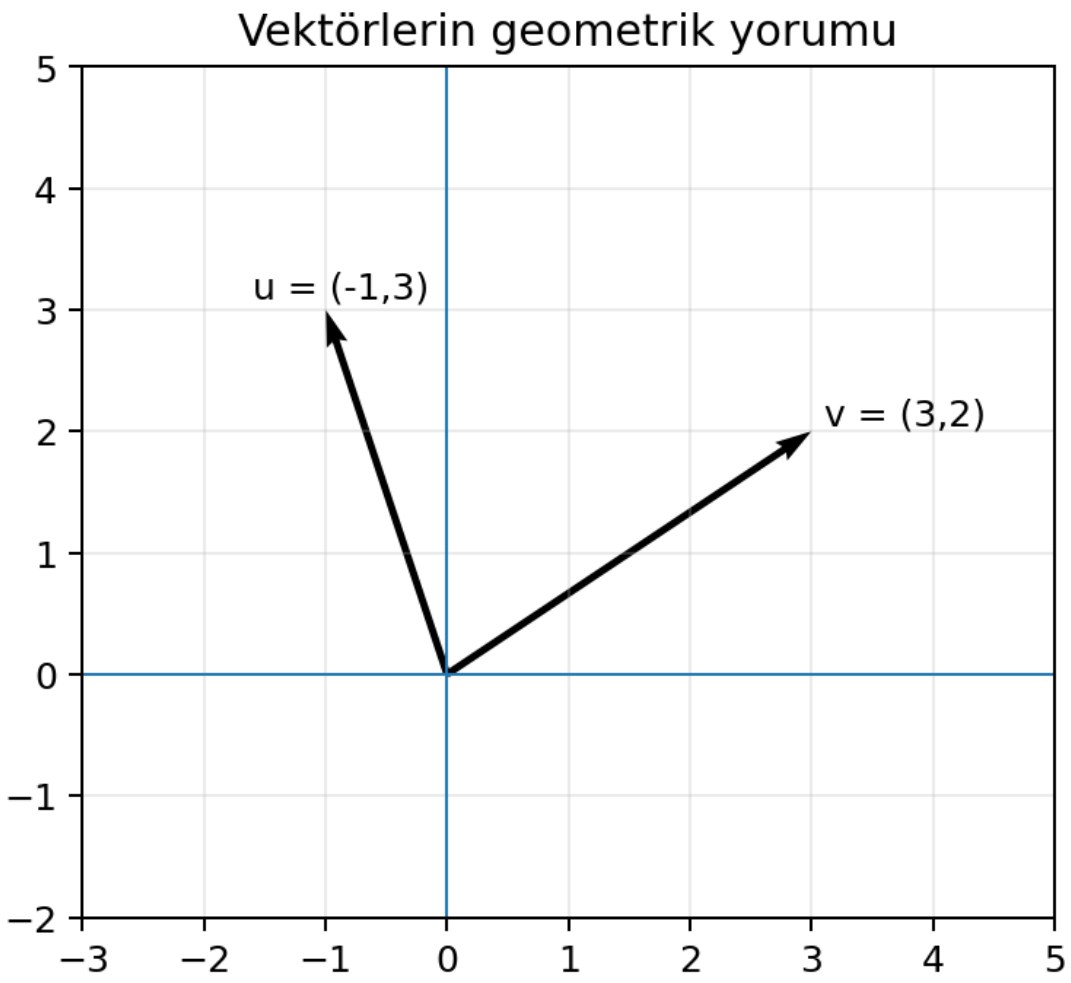
NumPy'da shape, lineer cebirdeki boyutların pratik karşılığıdır. Shape uyuşmazlıkları, gerçek projelerde lineer cebir hatalarının en görünür biçimidir.

```
import numpy as np


x = np.array([25, 50000, 1200])   # vector
X = np.array([[25, 50000, 1200],
              [31, 72000, 1600]]) # matrix


print(x.shape)   # (3,)
print(X.shape)   # (2, 3)
```

3. Vektörler: verinin geometrisi



Bir vektör hem bir sayı listesi hem de uzayda yön/büyüklük taşıyan geometrik bir nesnedir. Veri biliminde bu iki yorum birlikte kullanılır.

İşlem	Formül fikri	Yorum
Toplama	u + v	Feature temsillerini toplamak / displacement
Scalar çarpım	c v	Vektörün büyüklüğünü ölçeklemek
Norm	||v||	Vektörün uzunluğu
Dot product	u · v	Hizalanma / benzerlik ölçüsü
Distance	||u-v||	İki gözlem arasındaki uzaklık

3.1 Normlar

L2 norm (Euclidean norm) en yaygın uzunluk ölçüsüdür: sqrt(x1^2 + x2^2 + ... + xn^2). L1 norm ise mutlak değerlerin toplamıdır.

ML bağlantısı
L1 ve L2 regularization doğrudan bu norm fikirlerine dayanır. L1 bazı katsayıları sıfıra iterek sparsity üretirken, L2 katsayıları daha yumuşak biçimde küçültür.

4. Dot product, açı ve cosine similarity

İki vektörün dot product'ı, vektörlerin ne kadar aynı yönde olduğunu ölçer. u·v = ||u|| ||v|| cos(theta). Bu yüzden cosine similarity, metin embedding'lerinde ve recommendation sistemlerinde yaygındır.

```
u = np.array([1., 2., 3.])
v = np.array([2., 1., 3.])

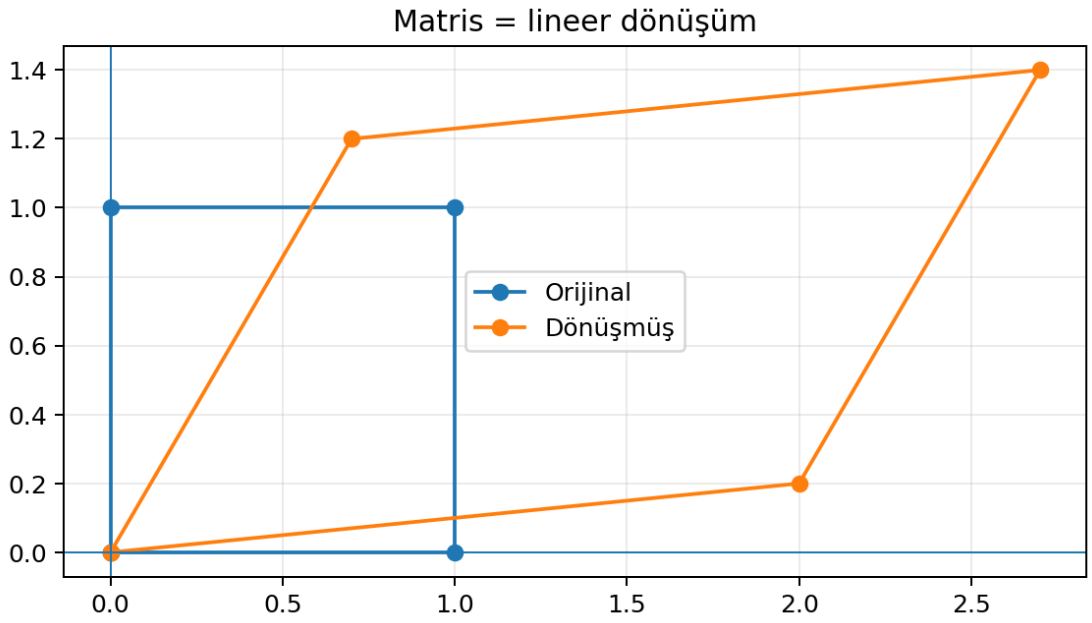
dot = u @ v
cos_sim = dot / (np.linalg.norm(u) * np.linalg.norm(v))

print(dot)
print(cos_sim)
```

Sezgi
Cosine similarity büyüklüğü değil yönü karşılaştırır. Bu nedenle iki embedding farklı normlara sahip olsa bile semantik yönleri benzerse cosine similarity yüksek olabilir.

5. Matrisler: veri tablosundan dönüşüme

Bir matrix satır ve sütunlardan oluşur. Veri biliminde çoğu zaman satırlar gözlem, sütunlar feature'dır. Ancak matris aynı zamanda bir lineer dönüşümü temsil eder.



Kavram	Örnek
Transpose	X.T ile satır/sütunları yer değiştirir.
Identity matrix	I, çarpıldığında nesneyi değiştirmez.
Diagonal matrix	Sadece köşegen elemanları anlamlıdır.
Symmetric matrix	A = A.T; covariance matrix önemli örnektir.

Sparse matrix	Elemanların çoğu 0; NLP ve recommender sistemlerinde sık görülür.

5.1 Matris çarpımı

A matrisi (m x n), B matrisi (n x p) ise AB tanımlıdır ve sonuç (m x p) boyutundadır. İç boyutların eşleşmesi gerekir.

```
A = np.array([[1, 2],
              [3, 4]])
B = np.array([[5, 6],
              [7, 8]])

C = A @ B
print(C)
```

En önemli shape kuralı
(m x n) @ (n x p) -> (m x p). NumPy'da @ operatörünü gördüğünde önce shape'leri yaz.

6. Lineer kombinasyon ve span

Bir vektör, başka vektörlerin katsayılarla çarpılıp toplanmasıyla oluşuyorsa lineer kombinasyondur. Bir vektör grubunun üretebildiği tüm noktaların kümesine span denir.

Örneğin iki bağımsız 2D vektör tüm düzlemi span edebilir. Ama aynı doğrultudaki iki vektör yalnızca bir doğruyu span eder.

ML bağlantısı
Linear regression, hedefi feature sütunlarının lineer kombinasyonu ile açıklamaya çalışır. Bu yüzden multicollinearity, feature'ların aynı yönleri tekrar tekrar temsil etmesi problemine dönüşür.

7. Lineer bağımsızlık, basis, dimension

Vektörlerden hiçbiri diğerlerinin lineer kombinasyonu olarak yazılamıyorsa küme lineer bağımsızdır. Basis, bir uzayı span eden ve gereksiz tekrar içermeyen bağımsız vektörler kümesidir.

Kavram	Sezgi	Veri bilimi karşılığı
Lineer bağımsızlık	Yeni yön / yeni bilgi	Redundant feature yok
Basis	Uzayı tanımlayan minimum yönler	Yeni koordinat sistemi
Dimension	Bağımsız yön sayısı	Efektif bilgi boyutu
Rank	Matrisin bağımsız bilgi miktarı	Feature redundancy ölçüsü

8. Rank ve multicollinearity

Bir matrisin rank'i, bağımsız satır veya sütun yönlerinin sayısıdır. Full rank olmayan design matrix, lineer regresyon gibi yöntemlerde katsayıların tekil biçimde belirlenmesini engelleyebilir.

```
X = np.array([[1, 2, 3],
              [2, 4, 6],
              [3, 6, 9]])


print(np.linalg.matrix_rank(X))  # 1
```

Gerçek proje örneği
total_price = unit_price * quantity gibi deterministik ilişkiler veya aynı kategorinin tüm dummy değişkenlerini intercept ile birlikte modele vermek rank sorununa yol açabilir.

9. Determinant

Determinant kare bir matrisin dönüşüm sırasında alanı/hacmi ne kadar ölçeklediğini anlatır. Determinant 0 ise dönüşüm uzayın bir boyutunu çökertmiştir ve matris invertible değildir.

```
A = np.array([[2, 1],
              [1, 3]])


print(np.linalg.det(A))
```

Ezber yerine sezgi
det(A)=0 -> bilgi bir boyutta çökmüş -> ters dönüşüm mümkün değil. Bu bağlantı inverse ve rank kavramlarını birleştirir.

10. Inverse matrix ve lineer denklem sistemleri

Ax=b denkleminde A invertible ise teorik olarak x=A^{-1}b yazılabilir. Ancak sayısal hesaplamada açıkça inverse almak yerine solve kullanmak daha doğru ve kararlıdır.

```
A = np.array([[2., 1.],
              [1., 3.]])
b = np.array([5., 7.])


x = np.linalg.solve(A, b)
print(x)
```

Senior alışkanlığı
Kodda np.linalg.inv(A) @ b yerine np.linalg.solve(A, b) tercih et. Büyük sistemlerde explicit inverse gereksiz maliyet ve sayısal hata yaratabilir.

11. Projection ve least squares

Projection, bir vektörü belirli bir alt uzaya en yakın noktaya taşımaktır. Linear regression'ın geometrik yorumu tam olarak budur: hedef y, X'in sütun uzayına projekte edilir.

Ordinary Least Squares, residual karelerinin toplamını minimize eder. Normal equation teorik olarak beta = (X^T X)^{-1} X^T y biçiminde yazılır; pratikte QR/SVD tabanlı çözücüler daha güvenlidir.

```
X = np.c_[np.ones(5), np.arange(5)]
y = np.array([1.0, 2.2, 2.9, 4.1, 5.2])


beta, *_ = np.linalg.lstsq(X, y, rcond=None)
print(beta)
```

Bağlantı
Regression katsayılarını sadece formül olarak değil, 'y'nin feature uzayına en iyi projection'ı' olarak düşünürsen residual, rank ve multicollinearity daha kolay oturur.

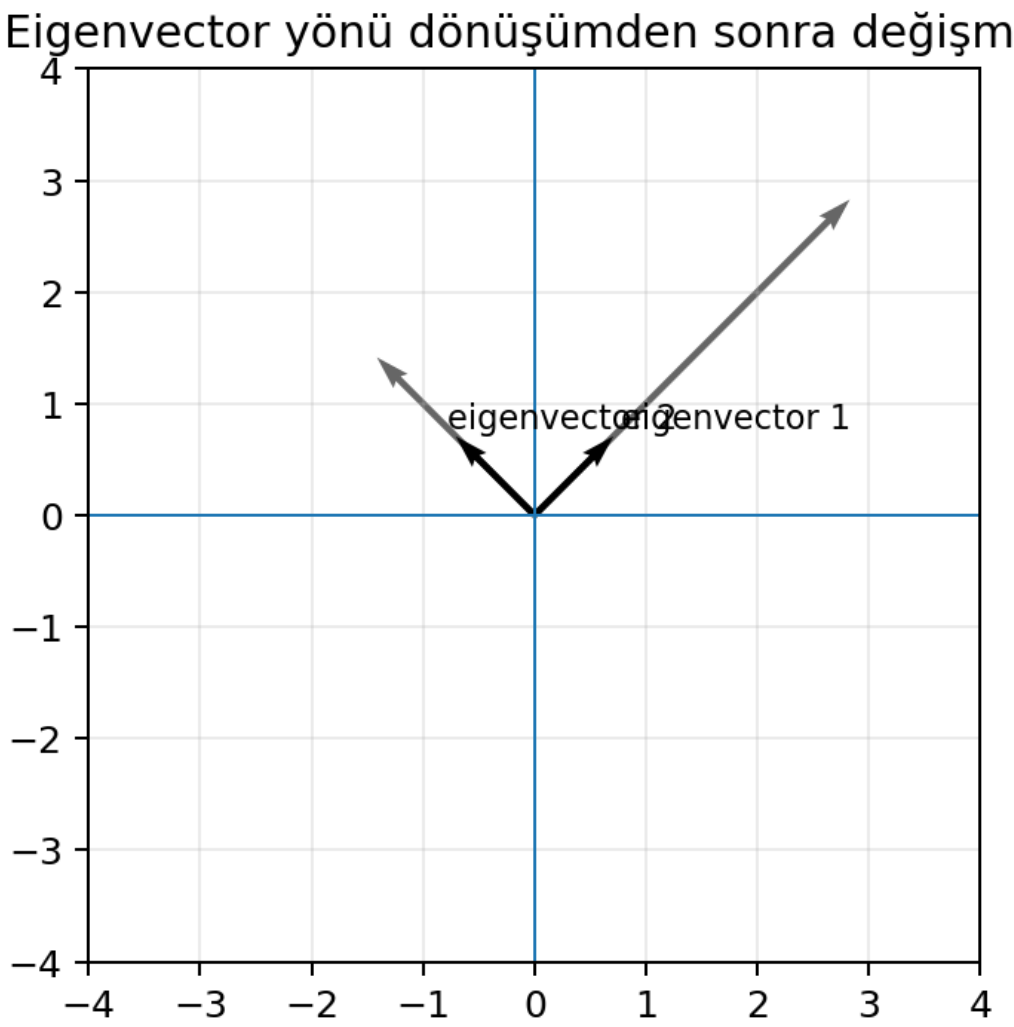
12. Orthogonality ve orthonormal basis

u·v = 0 ise vektörler orthogonal'dır. Uzunlukları 1 ise orthonormal'dır. Orthogonal eksenler bilgiyi birbirine karıştırmadan temsil etmeyi kolaylaştırır.

Nerede görülür?	Neden yararlı?
QR decomposition	Lineer sistemleri kararlı çözmek
PCA components	Birbirine dik ana yönler
Fourier benzeri basis'ler	Sinyali bağımsız bileşenlere ayırmak
Neural network initialization	Bazı yöntemlerde bilgi akışını korumak

13. Eigenvalue ve eigenvector

A v = lambda v eşitliğini sağlayan v vektörüne eigenvector, lambda değerine eigenvalue denir. Eigenvector dönüşüm altında yönünü korur; sadece ölçeklenir.



Covariance matrix için en büyük eigenvalue'a karşılık gelen eigenvector, verinin en fazla varyans gösterdiği yönü verir.

```
A = np.array([[3., 1.],
              [1., 3.]])


eigenvalues, eigenvectors = np.linalg.eig(A)
print(eigenvalues)
print(eigenvectors)
```

14. Covariance matrix

Covariance matrix feature'ların birlikte nasıl değiştiğini özetler. Köşegen elemanlar varyans, köşegen dışı elemanlar covariance'dır. Simetriktir.

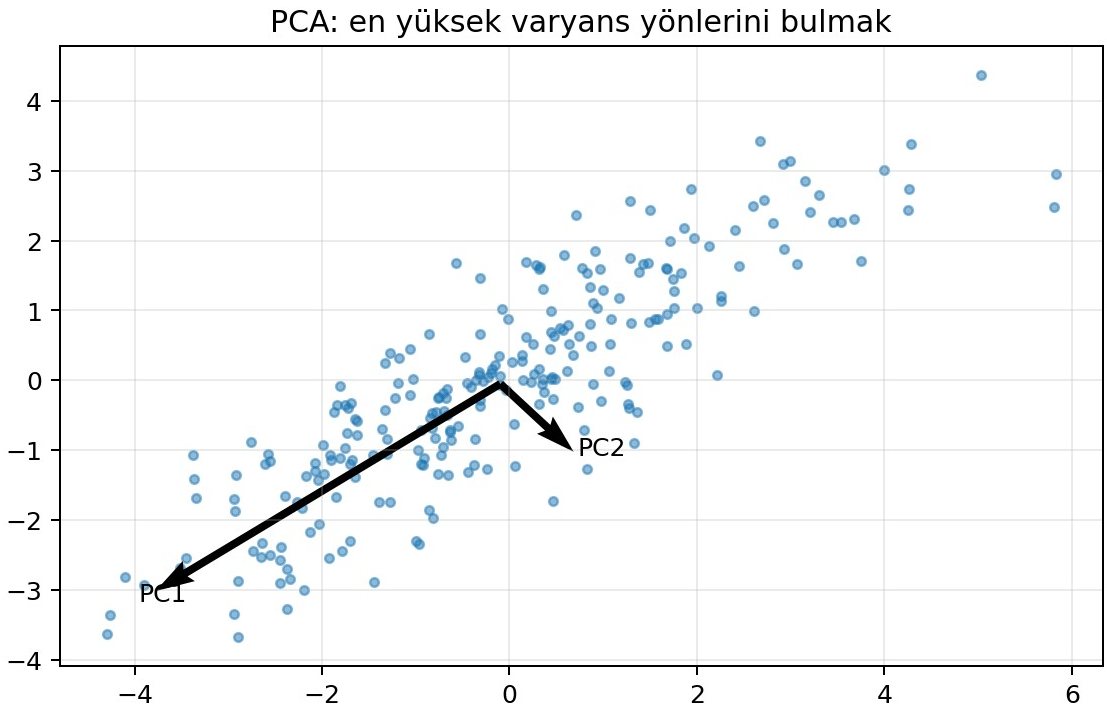
```
X = np.array([[1., 2.],
              [2., 2.5],
```

```
       [3., 4.],
       [4., 5.]])

cov = np.cov(X, rowvar=False)
print(cov)
```

PCA bağlantısı
PCA, merkezlenmiş verinin covariance matrix'indeki eigenvector'ları kullanarak yeni orthogonal eksenler bulur.

15. PCA'nın lineer cebir mantığı



1. Veriyi merkezle: her feature'dan ortalamasını çıkar.
2. Covariance matrix'i hesapla veya doğrudan SVD kullan.
3. Eigenvector / singular vector yönlerini bul.
4. En yüksek varyansı açıklayan ilk k yönü seç.
5. Veriyi bu yeni basis'e project et.

Önemli
PCA supervised değildir; target'ı bilmez. En fazla varyansı korur ama en fazla predictive bilgiyi koruyacağı garanti değildir.

16. SVD - Singular Value Decomposition

Her m x n matris için X = U Sigma V^T ayrışımı düşünülebilir. SVD, eigen decomposition'dan daha genel ve sayısal olarak çok güçlüdür.

Bileşen	Yorum
U	Sol singular vectors; gözlem uzayındaki yönler
Sigma	Singular values; yönlerin gücü/önemi
V	Sağ singular vectors; feature uzayındaki yönler

SVD; PCA, low-rank approximation, recommender systems, text processing ve numerical linear algebra için temel araçlardan biridir.

```
X = np.array([[1., 2.],
              [3., 4.],
              [5., 6.]])


U, s, Vt = np.linalg.svd(X, full_matrices=False)
print(s)
```

Low-rank approximation
En büyük k singular value ve karşılık gelen vektörleri tutarak matrisi daha az bilgiyle yaklaşık temsil edebilirsin. Bu fikir boyut indirgeme ve matrix factorization'ın temelidir.

17. Distance, similarity ve embeddings

Ölçü	Ne ölçer?	Kullanım
Euclidean distance	Düz çizgi uzaklığı	KNN, clustering
Manhattan distance	Koordinat bazlı toplam uzaklık	Bazı sparse/problem yapıları
Cosine similarity	Yön benzerliği	Text/image embeddings
Dot product	Hizalanma + büyüklük	Recommendation/ranking

Embedding'ler yüksek boyutlu vektörlerdir. Anlam benzerliği çoğu uygulamada bu vektörlerin geometrisi üzerinden ölçülür.

```
def cosine_similarity(a, b):
  return (a @ b) / (np.linalg.norm(a) * np.linalg.norm(b))
```

18. Matrix calculus'e kısa köprü

Bu modülün ana konusu calculus değildir; ancak modern ML'de gradient'ler vektör/matris biçiminde taşınır. Örneğin linear regression loss'u türetildiğinde X^T(Xw-y) yapısı çıkar.

Neden önemli?
Neural network'lerde forward pass matris çarpımlarıyla, backward pass ise bu matris işlemlerinin türevleriyle yürür. Lineer cebir sağlam değilse deep learning formülleri gereksiz derecede zor görünür.

19. NumPy ile lineer cebir - pratik kalıplar

```
import numpy as np

# dot / matrix multiply
a @ b
X @ w

# norm
np.linalg.norm(x)

# rank
np.linalg.matrix_rank(X)

# determinant
np.linalg.det(A)

# solve Ax=b
np.linalg.solve(A, b)

# least squares
np.linalg.lstsq(X, y, rcond=None)

# eigen decomposition
np.linalg.eigh(C)   # symmetric matrix için iyi seçim

# SVD
np.linalg.svd(X, full_matrices=False)
```

Hata	Daha iyi yaklaşım
Her yerde inverse almak	solve / lstsq kullan
Shape kontrol etmemek	İşlemden önce boyutları yaz
Feature scale'lerini unutmak	Distance/PCA öncesi ölçek düşün
Rank deficiency'yi görmezden gelmek	matrix_rank / condition number kontrol et
Eigen için genel solver kullanmak	Symmetric covariance'da eigh tercih et

20. Numerical stability ve condition number

Matematiksel olarak çözülebilir bir problem sayısal olarak kararsız olabilir. Condition number, küçük giriş hatalarının çözümü ne kadar büyütebileceğini özetler.

```
cond = np.linalg.cond(X)
print(cond)
```

Yorum
Condition number çok büyükse feature'lar neredeyse lineer bağımlı olabilir, ölçekler çok farklı olabilir veya problem intrinsically ill-conditioned olabilir.

21. Gerçek vaka: müşteri davranışında PCA

Bir e-ticaret şirketinde her müşteri için 20 davranış feature'ı olduğunu düşün: sipariş sayısı, ortalama sepet, indirim oranı, kategori çeşitliliği, ziyaret sıklığı, son alışverişten geçen gün vb.

6. Feature'ları standartlaştır.
7. Correlation ve redundancy'yi incele.
8. PCA uygula.
9. Explained variance ratio ile kaç component tutulacağını değerlendir.
10. Component loadings ile PC'lerin hangi feature kombinasyonlarını temsil ettiğini yorumla.
11. İndirgenmiş temsili clustering veya görselleştirme için kullan.

```
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

Xs = StandardScaler().fit_transform(X)

pca = PCA(n_components=0.90)
Z = pca.fit_transform(Xs)

print(pca.n_components_)
print(pca.explained_variance_ratio_)
```

İş yorumu
PCA sonucunu 'PC1 müşteri değeri, PC2 indirim hassasiyeti' gibi isimlendirmek ancak loadings bunu gerçekten destekliyorsa yapılmalıdır.

22. Mülakat soruları

Soru	Kısa cevap
Vector ile matrix arasındaki fark nedir?	Vector tek eksenli bir koordinat listesi; matrix satır-sütun yapılı 2D lineer cebir nesnesidir. Veri seti çoğunlukla matrix, tek gözlem vector olarak temsil edilir.
Dot product ne anlatır?	İki vektörün hizalanmasını ölçer; normlarla normalize edilirse cosine similarity elde edilir.
Rank neden önemlidir?	Matrisin bağımsız bilgi boyutunu gösterir. Rank deficiency multicollinearity ve tekil çözüm problemlerine işaret edebilir.

Determinant 0 ne demektir?	Kare matris invertible değildir; dönüşüm en az bir boyutu çökertmiştir.
Neden inverse yerine solve?	Daha az hesap maliyeti ve daha iyi numerical stability.
Eigenvector nedir?	Lineer dönüşüm uygulandığında yönü değişmeyen, sadece ölçeklenen vektördür.
PCA neden eigenvector kullanır?	Covariance matrix'in eigenvector'ları varyans yönlerini, eigenvalue'lar bu yönlerdeki varyans miktarını verir.
PCA ile feature selection aynı mı?	Hayır. PCA orijinal feature'ların kombinasyonlarından yeni component'ler üretir; feature selection orijinal feature'lardan alt küme seçer.
SVD neden önemli?	Her dikdörtgen matrise uygulanabilir, low-rank yapı çıkarır ve PCA/matrix factorization gibi yöntemlerin temelidir.
Cosine similarity ne zaman Euclidean'dan daha uygun?	Yönün büyüklükten daha önemli olduğu normalized/embedding temsillerinde.
Orthogonal ne demektir?	Dot product 0; geometrik olarak dik, bilgi eksenleri açısından bağımsız yön sezgisi verir.
Condition number ne söyler?	Problemin girişteki küçük perturbation'lara duyarlılığını; büyük değer instability uyarısıdır.

23. Alıştırmalar

12. İki 5-boyutlu vektör oluştur; dot product, L1/L2 norm ve cosine similarity hesapla.
13. 3x3 bir matris için determinant, rank ve inverse/solve sonuçlarını karşılaştır.
14. Bir feature'ı diğer iki feature'ın tam lineer kombinasyonu yap ve rank'in nasıl değiştiğini göster.
15. 2D bir veri setini StandardScaler sonrası PCA ile tek boyuta indir; explained variance'ı yorumla.
16. Bir covariance matrix için eigenvalue/eigenvector hesapla ve büyük eigenvalue yönünü görselleştir.
17. SVD ile rank-k approximation yap; k arttıkça reconstruction error'un nasıl düştüğünü incele.
18. KNN öncesinde feature scaling yapılmadığında Euclidean distance'ın nasıl bozulduğunu küçük bir örnekle göster.

24. Final cheat sheet

Kavram	Hatırlanacak ana fikir
Vector	Bir gözlem / yön / feature listesi
Matrix	Veri tablosu veya lineer dönüşüm
Dot product	Hizalanma ve similarity temeli
Norm	Uzunluk / regularization temeli
Rank	Bağımsız bilgi boyutu
Determinant	Hacim ölçeği; 0 ise inverse yok
Inverse	Dönüşümü geri çevirme; pratikte solve tercih edilir
Projection	Bir alt uzaydaki en yakın temsil
Orthogonality	Dik, birbirini karıştırmayan yönler
Eigenvector	Dönüşümde yönünü koruyan eksen
SVD	Genel matris ayrışımı ve low-rank yapı
PCA	En yüksek varyans yönlerine projection

Bu modülden sonra
Linear Regression, PCA, Recommendation Systems, Deep Learning ve embeddings konularındaki formülleri artık daha sezgisel okuyabilmelisin. Amaç elle determinant hesaplamak değil; modelin hangi uzayda ne yaptığını görebilmektir.